1η Εργαστηριακή Άσκηση

Ψηφιακά Συστήματα VLSI

Παναγιώτης Γερασιμόπουλος 03115208
personal@devcol.com

Ακαδημαϊκό Έτος 2025-2026

Contents

1. Μέρος Α 1
   1.1. Decoder 3-8 1
      1.1.1. Implementation 1
         1.1.1.1. Dataflow 1
         1.1.1.2. Behavioral 1
      1.1.2. Testbench 2
   1.2. Καταχωρητής Ολίσθησης 3
      1.2.1. Implementation 3
      1.2.2. Test Bench 4
   1.3. Μετρητής 6
      1.3.1. Implementation 6
      1.3.2. Test Bench 8
2. Μέρος Β 10

1. Μέρος Α

1.1. Decoder 3-8

Ο αποκωδικοποιητής 3 σε 8 είναι ένα συνδυαστικό κύκλωμα που για κάθε συνδυασμό των εισόδων του ενεργοποιεί μόνο μία από τις εξόδους.

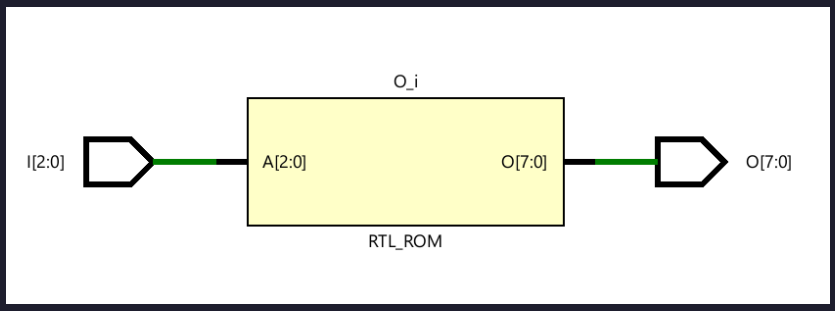

Figure 2: RTL

1.1.1. Implementation

Ο πίνακας αληθείας του αποκωδικοποιητή φαίνεται από τις γραμμές 16-23 του κωδικά για το behavioral.

1.1.1.1. Dataflow

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Decoder3x8 is
 port (
   I : in  std_logic_vector(2 downto 0);
   O : out std_logic_vector(7 downto 0));
end entity;

architecture Dataflow of Decoder3x8 is
begin
  with I select O <=
  "00000001"      when "000",
  "00000010"      when "001",
  "00000100"      when "010",
  "00001000"      when "011",
  "00010000"      when "100",
  "00100000"      when "101",
  "01000000"      when "110",
  "10000000"      when "111",
  (others => '-') when others;
end Dataflow;
```

1.1.1.2. Behavioral

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity Decoder3x8 is
 port (
   I : in  std_logic_vector(2 downto 0);
   O : out std_logic_vector(7 downto 0));
end entity;

architecture Behavioral of Decoder3x8 is
begin
  process(I)
  begin
    case I is
      when "000"  => O <= "00000001";
      when "001"  => O <= "00000010";
      when "010"  => O <= "00000100";
      when "011"  => O <= "00001000";
      when "100"  => O <= "00010000";
      when "101"  => O <= "00100000";
      when "110"  => O <= "01000000";
      when "111"  => O <= "10000000";
      when others => O <= (others => '-');
    end case;
  end process;
end Behavioral;
```

1.1.2. Testbench

Το κύκλωμα έχει πολύ λίγες πιθανές εισόδους και δεν έχει state οπότε μπορούμε εύκολα να σχεδιάσουμε εξαντλητικά test bench όπως φαίνεται παρακάτω.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Bench is
end entity;

architecture Behavioral of Bench is
  component Decoder3x8 is
  port (
    I   : in  std_logic_vector(2 downto 0);
    O   : out std_logic_vector(7 downto 0)
  );
  end component;
signal I: std_logic_vector(2 downto 0);
signal O: std_logic_vector(7 downto 0);

begin
  uut: Decoder3x8 port map (I => I, O => O);

  process begin

    for j in 0 to 7 loop
      I <= std_logic_vector(to_unsigned(j, 3));
      wait for 10 ns;
```

```
      end loop;
    wait;

  end process;
end architecture;
```

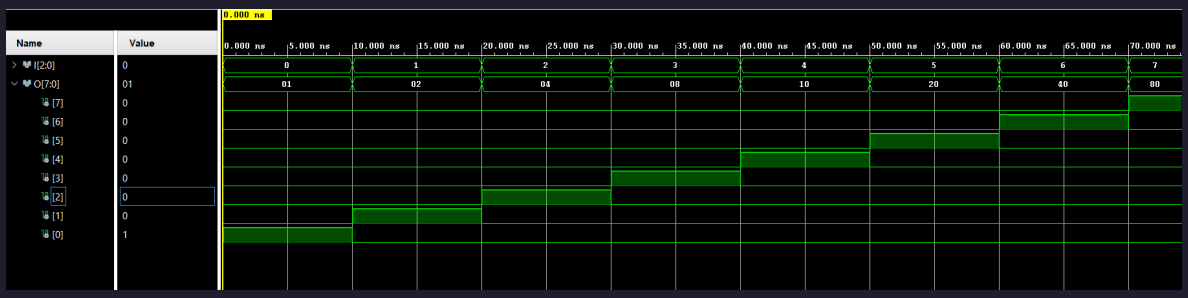

Figure 3: Test Bench

1.2. Καταχωρητής Ολίσθησης

Ο καταχτητής ολίσθησης είναι σύγχρονο κύκλωμα που δέχεται μια παράλληλη είσοδο των 4bit μέσω του I και μια συριακή μέσω του SerIn. Τα σήματα έλεγχου του είναι το reset που τον επαναφέρει στο 0 ασύγχρονα, το load που ελέγχει την παράλληλη είσοδο, το enable που ελέγχει την σειριακή λειτουργία του κυκλώματος και το slide bit που ελέγχει την κατεύθυνση της ολίσθησης ασύγχρονα.

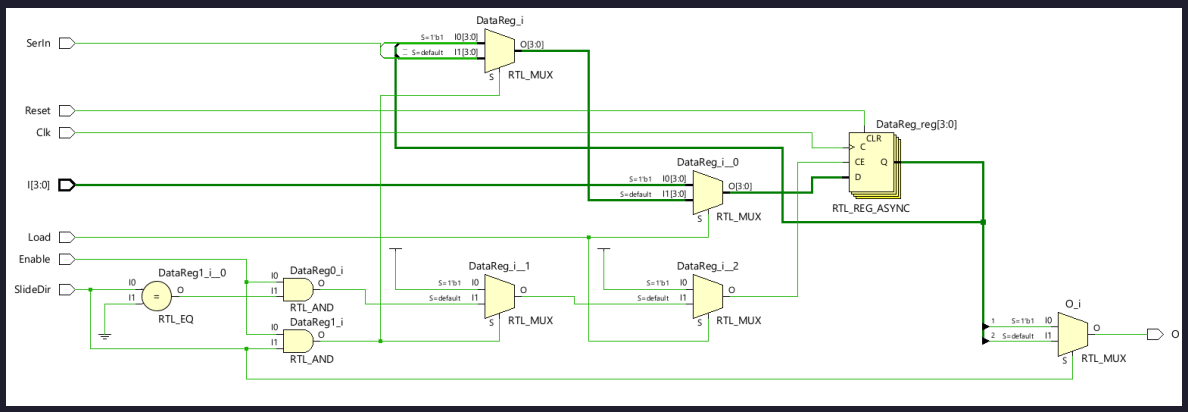

Figure 4: RTL

1.2.1. Implementation

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ShiftRegister is
  Port(
    Clk, Reset, Load, Enable, SerIn, SlideDir: in std_logic;
    I : in std_logic_vector(3 downto 0);
    O : out std_logic
  );
end entity;
```

```
architecture zzz of ShiftRegister is
  signal DataReg: std_logic_vector(3 downto 0);
begin

  edge:process(Clk, Reset) begin
    if Reset = '1' then
      DataReg <= (others => '0');
    elsif rising_edge(Clk) then
      if Load = '1' then
        DataReg <= I;
      elsif Enable = '1' and SlideDir = '1' then
        DataReg <= SerIn & DataReg(3 downto 1);
      elsif Enable = '1' and SlideDir = '0' then
        DataReg <= DataReg(2 downto 0) & SerIn;
      end if;
    end if;
  end process;

  O <= DataReg(0) when SlideDir = '1' else DataReg(3);

end zzz;
```

Το οποίο μας δίνει στο critical path του synthesis 5.351ns.

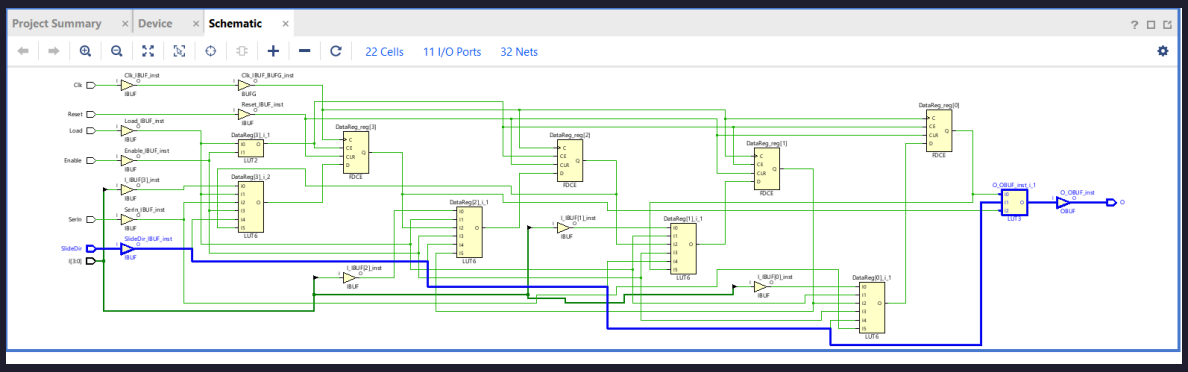

Figure 5: Critical Path 5.351ns

1.2.2. Test Bench

Φορτώνουμε τον αριθμό '1010' μέσω του load και κάνουμε αριστερό slide για 3 cycles φορτώνοντας 4 '1' μέσω του SerIn και μετά κάνουμε δυο cycles slide δεξιά περιμένοντας να δούμε τους άσους που φορτώσαμε στα register. Τα input δίνονται στο falling edge ενώ το σύστημα κάνει update στο rising edge.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Tb is
end entity;

architecture zzz of Tb is
  component ShiftRegister
   Port(
     Clk, Reset, Load, Enable, SerIn, SlideDir: in std_logic;
     I: in std_logic_vector(3 downto 0);
```

```
     O: out std_logic
    );
  end component;

  signal Clk:       std_logic := '0';
  signal Load:      std_logic;
  signal Reset:     std_logic;
  signal Enable:    std_logic;
  signal SlideDir:  std_logic;
  signal SerIn:     std_logic;
  signal I:         std_logic_vector(3 downto 0);
  signal O:         std_logic;
  constant CLOCK_PERIOD : time := 10 ns;

begin
  Dut: ShiftRegister port map (
    Clk       => Clk,
    Load      => Load,
    Reset     => Reset,
    Enable    => Enable,
    SlideDir  => SlideDir,
    SerIn     => SerIn,
    I         => I,
    O         => O
  );

  Clk <= not Clk after CLOCK_PERIOD / 2;

  Test: process begin
    Load <= '1';
    Reset <= '0';
    Enable <= '1';
    SlideDir <= '0';
    SerIn <= '1';
    I <= "1010";

    wait until falling_edge(Clk);
    Load <= '0';
    wait until falling_edge(Clk);
    wait until falling_edge(Clk);
    wait until falling_edge(Clk);
    SerIn <= '0';
    SlideDir <='1';
    wait until falling_edge(Clk);
    wait until falling_edge(Clk);
    Reset<='1';

    wait;
  end process;
end architecture;
```

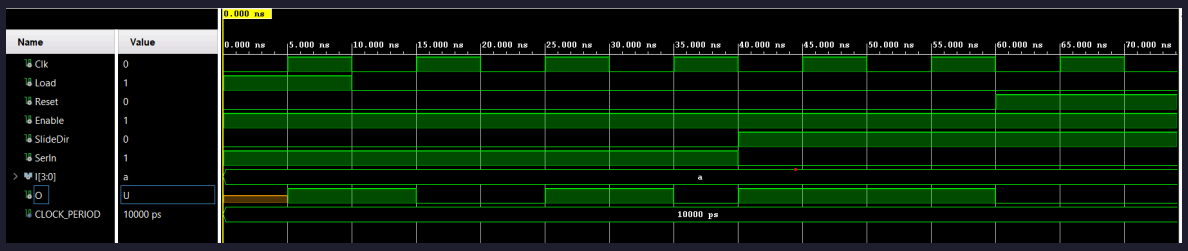

Figure 6: Test Bench

1.3. Μετρητής

Ο Μετρητής είναι ένα σύγχρονο κύκλωμα που μετράει σε διάδικο τους κύκλους του ρολογιού και κάνει wrap στο overflow, στην περίπτωση μας μετά τον αριθμό 7. Το ResetN μηδενίζει την έξοδο ασύγχρονα, το down ελέγχει αν ο μετρητής θα αυξάνει η μειώνει σε κύκλο, και το CountEn λειτουργεί ως start stop της μέτρησης. Επίσης προσδέθηκαν τα Modulo και Up που ελέγχουν το σημείο που κάνουμε wrap και την κατεύθυνση της μέτρησης.

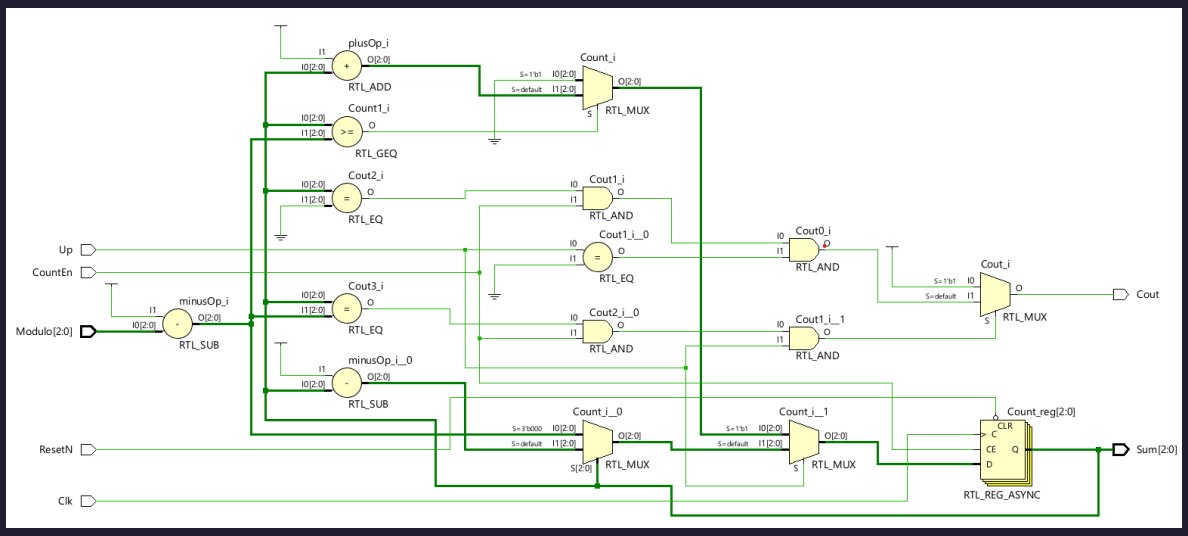

Figure 7: RTL

1.3.1. Implementation

Και τα δυο μέρη της άσκησης έγιναν σε ένα κύκλωμα για ευκολία στο debug, το κύκλωμα υποστηρίζει dynamic αλλαγή της τιμής του modulo αλλά μόνο προς το θετικό count (αφού δεν ζητήθηκε modulo για μέτρηση προς τα κάτω).

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Counter is
  Port (
    Clk,
    ResetN,
    Up,
    CountEn : in std_logic;
    Modulo : in std_logic_vector(2 downto 0);
```

```
    Sum : out std_logic_vector(2 downto 0);
    Cout : out std_logic
  );
end Counter;

architecture zzz of Counter is
signal Count : unsigned(2 downto 0);
begin

  process(Clk, ResetN) begin
    if ResetN = '0' then
      Count <= (others =>'0');
    elsif  (rising_edge(Clk)) then
      if CountEn = '1' then

        if Up = '1' then
          -- Up Count
          if Count >= unsigned(Modulo) - 1 then
            Count <= (others => '0');
          else
            Count <= Count + 1;
          end if;

        else
          -- Down Count
          if Count = 0 then
            Count <= unsigned(Modulo) - 1;
          else
            Count <= Count - 1;
          end if;
        end if;

      end if;
    end if;
  end process;

  Sum <= std_logic_vector(Count);
  Cout <= '1' when (Count = unsigned(Modulo)-1 and CountEn='1' and Up='1')
else
          '1' when (Count = 0 and CountEn = '1' and Up = '0') else
          '0';

end architecture;
```

Το οποίο μας δίνει στο critical path του synthesis 5.924ns.

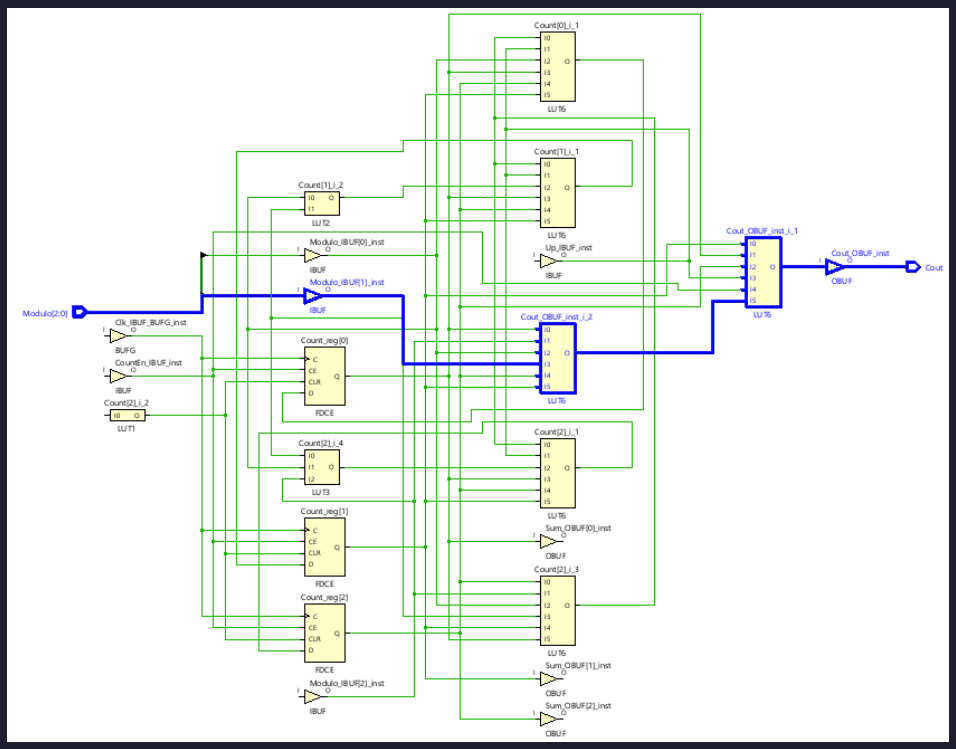

Figure 8: Critical Path 5.924ns

1.3.2. Test Bench

Για το test bench κάνουμε αρχικά μέτρηση προς τα πάνω μέχρι overflow και μετά προς τα κάτω μέχρι underflow. Στην συνέχεια αλλάζουμε το modulo και δοκιμάζουμε την δυναμική αλλαγή του στην μέτρηση προς τα πάνω.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Counter_Tb is
end Counter_Tb;

architecture Simulation of Counter_Tb is
  component Counter
    Port (
      Clk     : in std_logic;
      ResetN  : in std_logic;
      Up      : in std_logic;
      CountEn : in std_logic;
      Modulo  : in std_logic_vector(2 downto 0);
      Sum     : out std_logic_vector(2 downto 0);
      Cout    : out std_logic
    );
  end component;

  signal Clk     : std_logic := '0';
  signal ResetN  : std_logic := '0';
  signal Up      : std_logic := '1';
  signal CountEn : std_logic := '0';
  signal Modulo  : std_logic_vector(2 downto 0) := "101";
```

```
  signal Sum     : std_logic_vector(2 downto 0);
  signal Cout    : std_logic;
  constant CLK_PERIOD : time := 10 ns;

begin
  DUT: Counter port map (
    Clk     => Clk,
    ResetN  => ResetN,
    Up      => Up,
    CountEn => CountEn,
    Modulo  => Modulo,
    Sum     => Sum,
    Cout    => Cout
  );

  Clk <= not Clk after CLK_PERIOD / 2;

  process begin
    ResetN <= '0';
    wait for CLK_PERIOD * 2;
    ResetN <= '1';
    -- NOTE(acol): Up count with wrap
    CountEn <= '1';
    Up <= '1';
    Modulo <= "101";
    wait for CLK_PERIOD * 7;
    -- NOTE(acol): Down count with wrap
    Up <= '0';
    wait for CLK_PERIOD * 4;
    -- NOTE(acol): Modulo dynamic change
    Modulo <= "011";
    Up <= '1';
    wait for CLK_PERIOD * 5;
    -- NOTE(acol): Disable
    CountEn <= '0';
    wait for CLK_PERIOD * 3;

    wait;
  end process;
end Simulation;
```

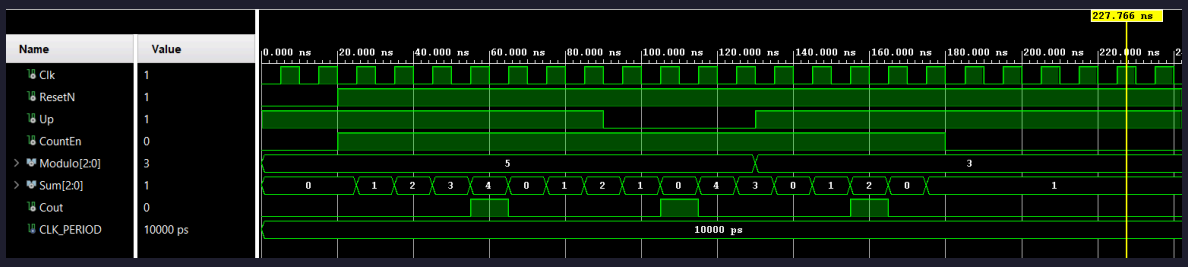

Figure 9: Test Bench

2. Μέρος Β